KAP CLI — Getting Started

This is a **starter project** that shows how Python can talk to the KAP (Borsa İstanbul) API and how you can build small research utilities by chaining those calls. The point is **not** to use this exact tool as-is for all your work — it is a base you **clone, run, and then extend by telling Claude Code what you want**. No programming background needed; you describe what you need in plain language and Claude Code writes the Python for you.

---

1. Before you start

Make sure you have these four things in place before running any commands.

Python 3.11 or newer Check: open a terminal (search for "Command Prompt" in the Start menu) and type:

```
python --version
```

You need to see `Python 3.11.x` or higher. If you don't have it: https://www.python.org/downloads/ — tick "**Add Python to PATH**" during install (important!).

Git Check:

```
git --version
```

If you don't have it: https://git-scm.com/download/win — install with the default options.

Claude Code You'll use this later to add your own commands. Get it from https://claude.ai/code if you haven't already.

A KAP API key + allowlisted network You need a key from MKK and your computer must be on a network that MKK has approved (allowlisted). Without this, commands will time out. If you're unsure, contact your IT team or KAP support: `kapdestek@mkk.com.tr`.

---

2. Get it running

Follow these steps in order. Every step tells you what success looks like.

Step 1 — Make a folder to keep the project in and go into it:

```
mkdir C:\kap
cd C:\kap
```

You'll know it worked: your terminal prompt now shows `C:\kap>`.

Step 2 — Download (clone) the project:

```
git clone https://github.com/ufuk-kilinc/kap-cli.git
```

You'll know it worked: a new `kap-cli` folder appears inside `C:\kap`.

Step 3 — Go into the project folder:

```
cd kap-cli
```

Step 4 — Install it (this sets up the `kap` command from the code you just downloaded):

```
pip install -e .
```

You'll know it worked: the output ends with `Successfully installed kap-cli` (and a version number).

Step 5 — Add your API key. The key must live in a file at `C:\Users\<YOUR-USERNAME>\.kap\.env`. The easiest way, from inside the `kap-cli` folder:

```
mkdir %USERPROFILE%\.kap
copy .env.example %USERPROFILE%\.kap\.env
```

Then open `%USERPROFILE%\.kap\.env` in Notepad (press Win + R, type `notepad %USERPROFILE%\.kap\.env`, press Enter) and set the two lines:

```
KAP_ACTIVE_ENV=production
KAP_PROD_API_KEY=paste-your-key-here
```

Save the file. This file stays on your computer only — it is never shared or uploaded.

Step 6 — Run it:

```
kap members
```

You'll know it worked: a table of companies prints in the terminal. ✓

> **Tip:** if `kap` is not recognized after installation, close the terminal window and open a new one (the PATH variable needs a refresh). Alternatively, `python -m kap members` always works without any PATH change.

---

3. A few things to try

Once `kap members` works, try these to see what else is available:

```
kap funds
```

Lists all funds in the KAP system.

```
kap last-index
```

Shows the latest disclosure index number market-wide (a handy freshness check).

```
kap fund-portfolio-pdf BON --out-dir C:\kap\reports
```

Downloads the latest portfolio allocation report PDF for fund code `BON` into the `C:\kap\reports` folder. The tool names the file for you and prints the full path where it saved it.

```
kap --help
```

Shows every available command with a one-line description.

> **Note on `--out` and `--out-dir`:** you can point either flag at a folder; the tool picks a sensible filename automatically and tells you exactly where it saved the file.

---

4. How it's built (so you can extend it)

There are two layers:

Raw API commands — one command per KAP endpoint. Each one makes a single call to the API and shows you the result. These are the basic building blocks.

Utilities — combine several raw calls into something more useful. For example, `fund-portfolio-pdf` finds the fund, locates its portfolio report, then downloads the PDF attachment — three calls, one command. The utilities shipped here are **demonstrations** of the kind of thing you can build. Your own research needs will be different, so you will build your own.

---

5. Make it your own — with Claude Code

This is how you add new commands without writing any code yourself. Open the `kap-cli` folder in Claude Code, then:

First, paste this so Claude Code fully understands the project:

> "Read docs/kap-cli-handoff.md and the README so you fully understand this project, then wait for my request."

Then describe what you want. For example:

> "Add a utility called `fund-nav` that fetches the latest net asset value for a given fund code, following the existing utility pattern (resolve the fund → call the right endpoint → show a table). Add tests and update the docs."

> "Add a command that lists all disclosures for a company on a given day."

Claude Code already knows the project's structure and rules from the built-in memory file (`docs/kap-cli-handoff.md`), so it writes the Python for you — you just describe what you need in plain language.

---

6. If something doesn't work

`kap` not recognized Close and reopen the terminal, then try again. Or use `python -m kap ...` as a drop-in replacement for any command (e.g. `python -m kap members`).

A command hangs for a while, then shows a connection error You are probably not on a network that MKK has approved. Try from your office network, or ask IT / `kapdestek@mkk.com.tr` to have your IP added to the allowlist.

"Missing API key" or similar config error Open `%USERPROFILE%\.kap\.env` in Notepad and check that:

- `KAP_ACTIVE_ENV=production` is set (not commented out)
- `KAP_PROD_API_KEY=` has your actual key after the = sign (no quotes, no extra spaces)

Save the file and try again.